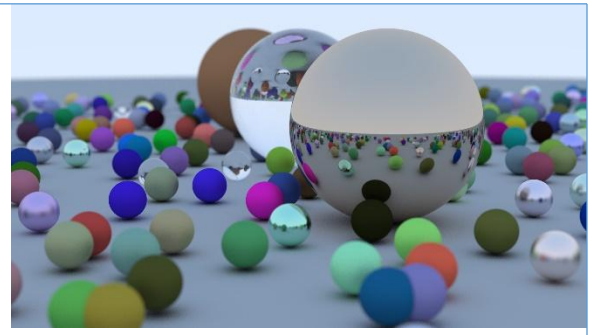
Comp4422



Computer Graphics

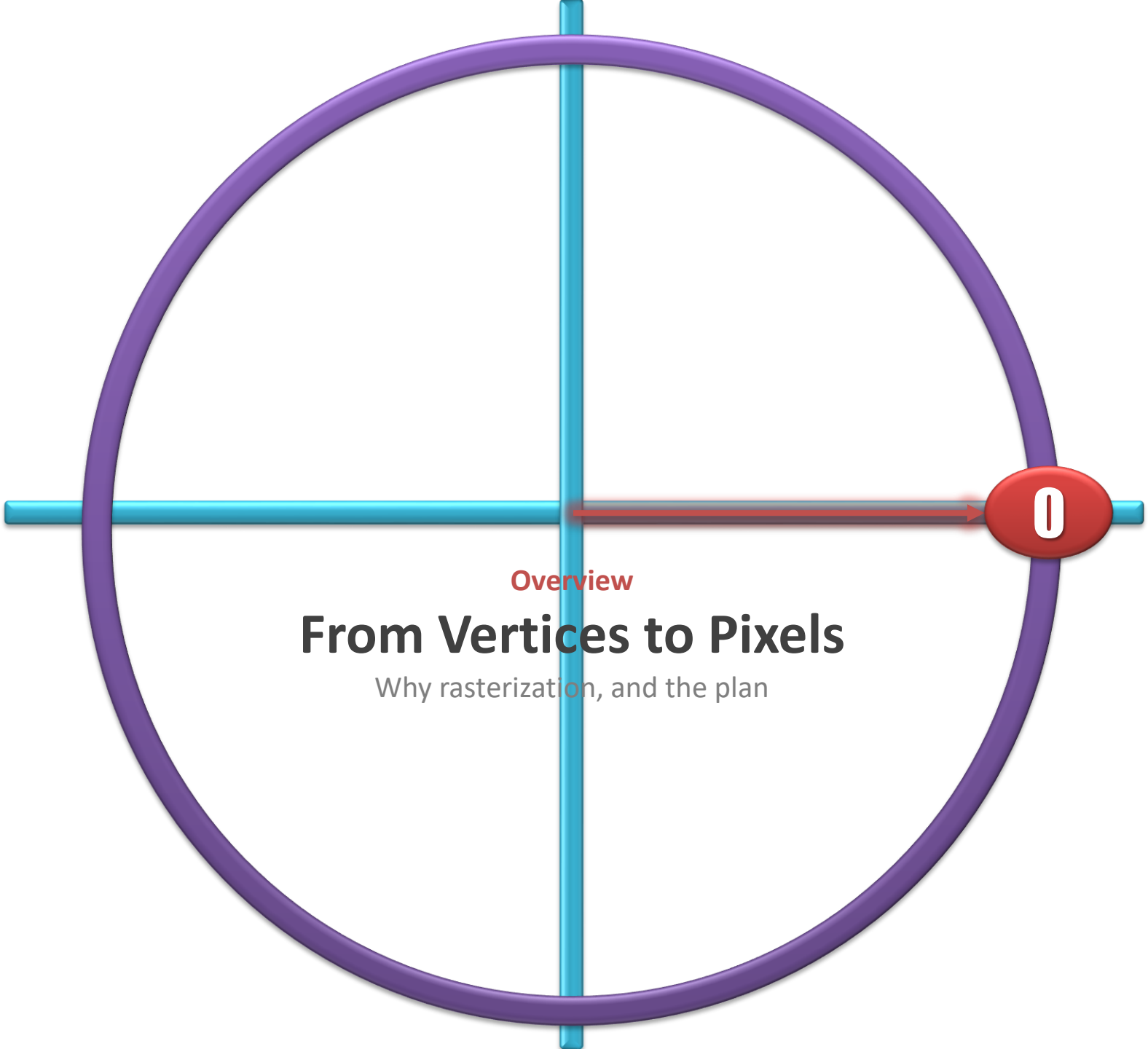
Lecture 05:

Rasterization & Visibility



Rasterization & Visibility

- Getting pixels on the screen
- Drawing lines: DDA and Bresenham
- Filling triangles: scanline and barycentric
- Visibility: painter's algorithm and the z-buffer



Overview

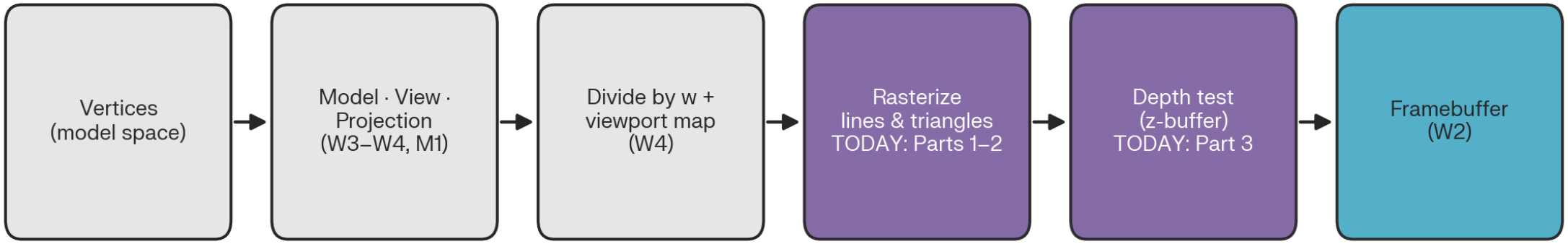
From Vertices to Pixels

Why rasterization, and the plan

Why This Lecture, Why Now

- In previous weeks, we have discussed:
  - Week 3: the matrices that move vertices.
  - Week 4: the camera that views them.
- The output of the vertex stage is triangles in SCREEN coordinates — but a screen is a grid of pixels
- **Today: which pixels does a triangle cover, and which triangle wins when they overlap?**
- Solution: filled triangles + z-buffer

Where Rasterization Fits



Weeks 3 to 4 gave us screen-space triangles. Today: which pixels they cover (Parts 1–2) and which surface is visible (Part 3).

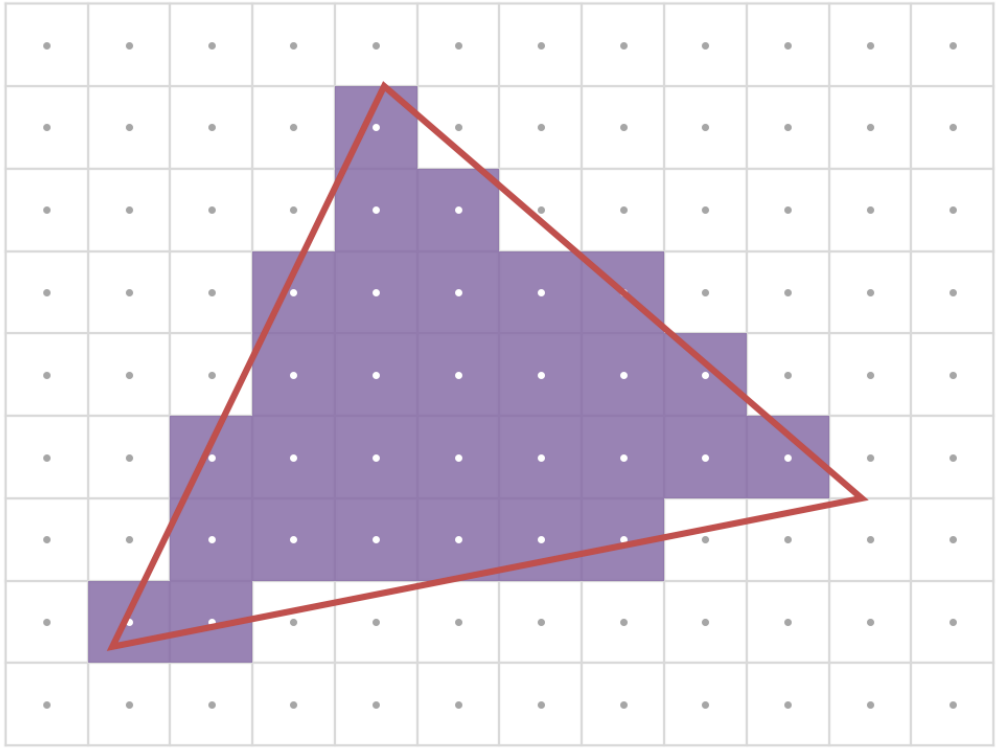
In your project, the above workflow could be one of your milestones, to be implemented on the CPU.

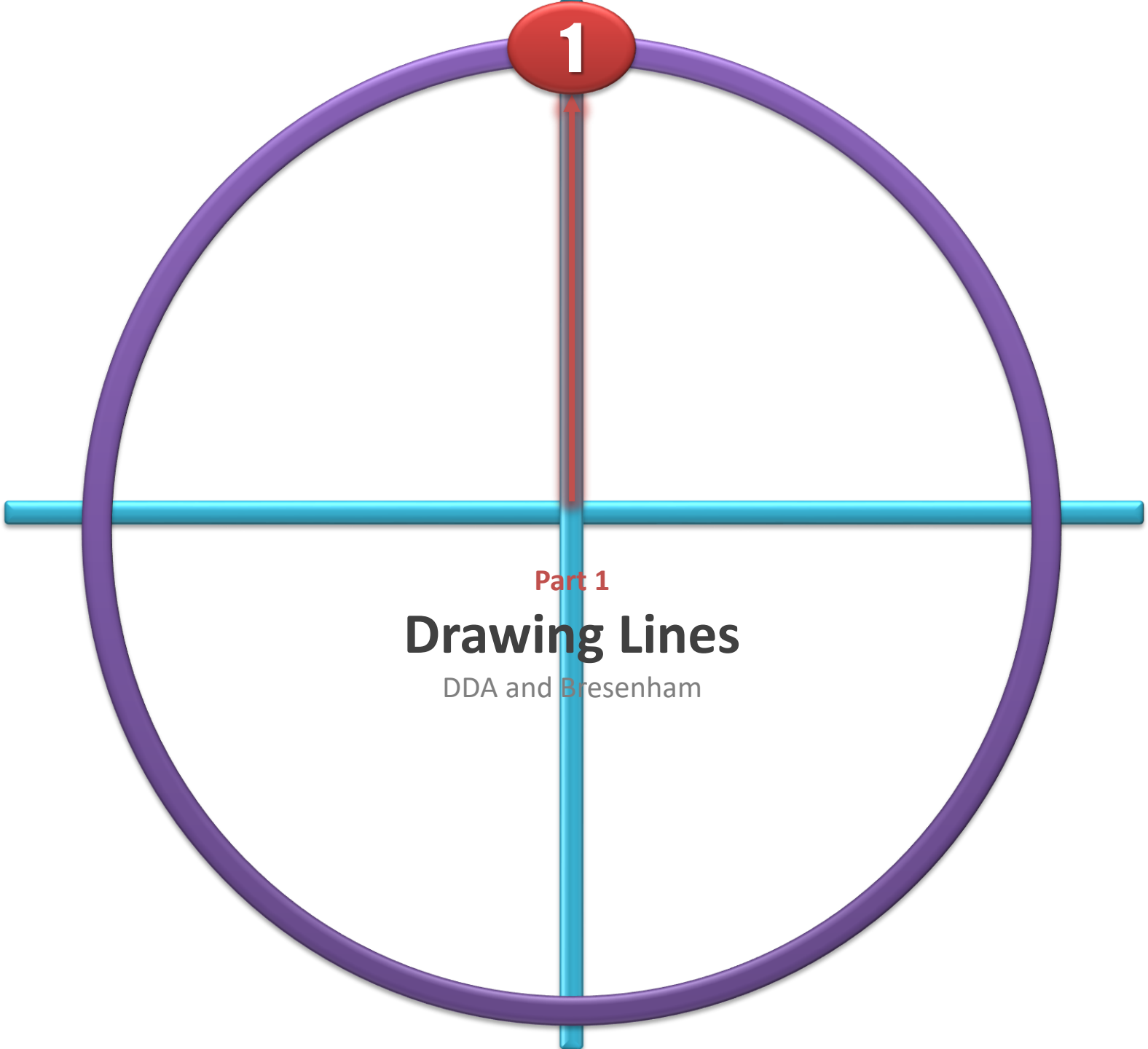
Roadmap

- Part 1: Drawing lines — DDA and Bresenham
- Part 2: Filling triangles — scanline fill and barycentric coordinates
- Part 3: Visibility — painter's algorithm vs the z-buffer
- Wrap-up: putting it together for the Lab exercise

The Rasterization Problem

- Geometry is continuous: a line has infinitely many points
- A screen is a discrete grid: we may only light whole pixels
- Rasterization = choosing which pixels best approximate the shape
- Convention: pixel (x, y) covers the square around its centre at (x + 0.5, y + 0.5)
- **Every algorithm today answers one question per pixel: in or out?**





Part 1

Drawing Lines

DDA and Bresenham

What Makes a Good Line Algorithm?

- Pixels should lie as close as possible to the ideal line
- Constant apparent brightness and thickness at any slope
- No gaps, no duplicate pixels; shared endpoints must not leave seams
- FAST: lines are drawn millions of times per second — avoid multiplies, divides and floats where possible

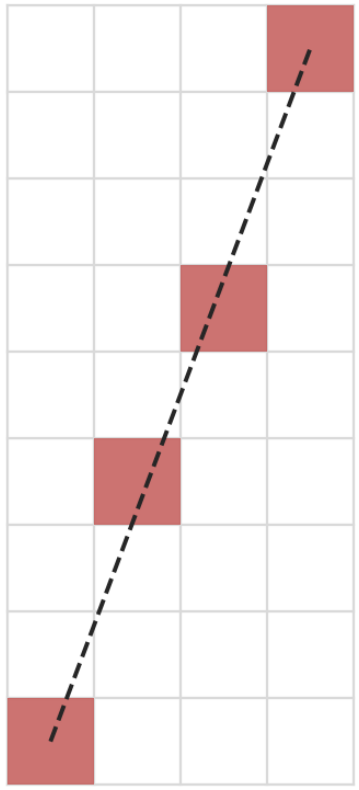
Attempt 1: Use the Line Equation

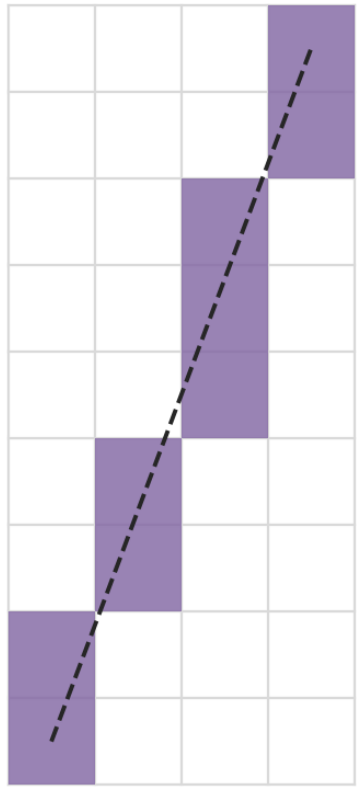
Slope-intercept form $y = m \cdot x + b$ — loop over x, compute y, round

```
function lineNaive(x0, y0, x1, y1, color) {
  const m = (y1 - y0) / (x1 - x0);
  const b = y0 - m * x0;
  for (let x = x0; x <= x1; x++) {
    const y = Math.round(m * x + b);   // one multiply + add + round PER PIXEL
    putPixel(img, x, y, ...color);
  }
}
```

Why Attempt 1 Fails

- Vertical line: x1 − x0 = 0 → division by zero
- Steep line (|m| > 1): stepping x by 1 moves y by more than 1 → gaps between pixels
- Float multiply + round for every pixel: slow, with rounding effects
- **Fix 1: step along the LONGER axis, so the other axis moves at most 1 per step**
- **Fix 2: stop multiplying — just ADD a constant increment each step**





Attempt 2: DDA (Digital Differential Analyzer)

Step along the longer axis; add constant increments

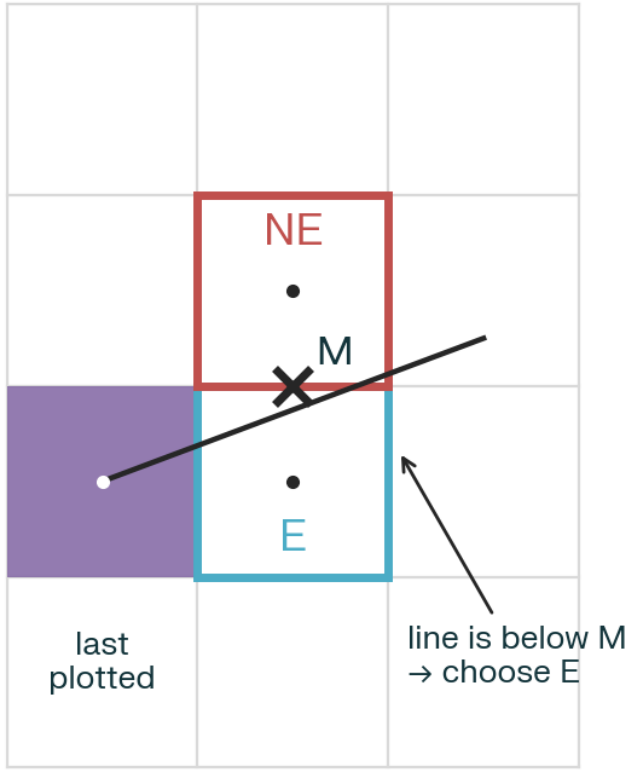
```
function lineDDA(x0, y0, x1, y1, color) {
  const dx = x1 - x0, dy = y1 - y0;
  const steps = Math.max(Math.abs(dx), Math.abs(dy));   // the longer axis
  const xInc = dx / steps, yInc = dy / steps;           // one is ±1, the other |.| <= 1
  let x = x0, y = y0;
  for (let i = 0; i <= steps; i++) {
    putPixel(img, Math.round(x), Math.round(y), ...color);
    x += xInc;  y += yInc;                               // additions only
  }
}
```

DDA: Good, But Not Great

- Works for every slope and direction — no gaps
- Still uses floating-point adds and Math.round for every pixel
- Rounding errors accumulate along long lines (xInc is rarely exact in binary)
- **Can we decide "which pixel next?" with INTEGER arithmetic only? Yes — Bresenham (1965)**

Bresenham: The Big Idea

- Take a line with 0 ≤ slope ≤ 1 and step x by one each time
- At each step the line lies between two candidate pixels: East (same y) or North-East (y + 1)
- Keep a decision term: how far has the ideal line drifted from the pixel centre?
- Drifted more than half a pixel upward → step NE; otherwise step E
- **One integer comparison per pixel — no multiplies, no divides, no floats**



Bresenham: Deriving the Decision Variable

- Line: $y = (dy/dx) \cdot x + B$, with $dx = x1 - x0$, $dy = y1 - y0$ and $0 \le dy \le dx$
- Multiply by dx and rearrange: $F(x, y) = dy \cdot x - dx \cdot y + dx \cdot B = 0$ on the line
- $F > 0$ on one side of the line and $F < 0$ on the other: the sign says which side a point is on
- Evaluate F at the midpoint between E and NE: its sign picks the closer pixel
- Incremental update: step E changes the decision value by $2 \cdot dy$; step NE changes it by $2 \cdot (dy - dx)$
- **Initial value $d = 2 \cdot dy - dx$; thereafter only integer updates**

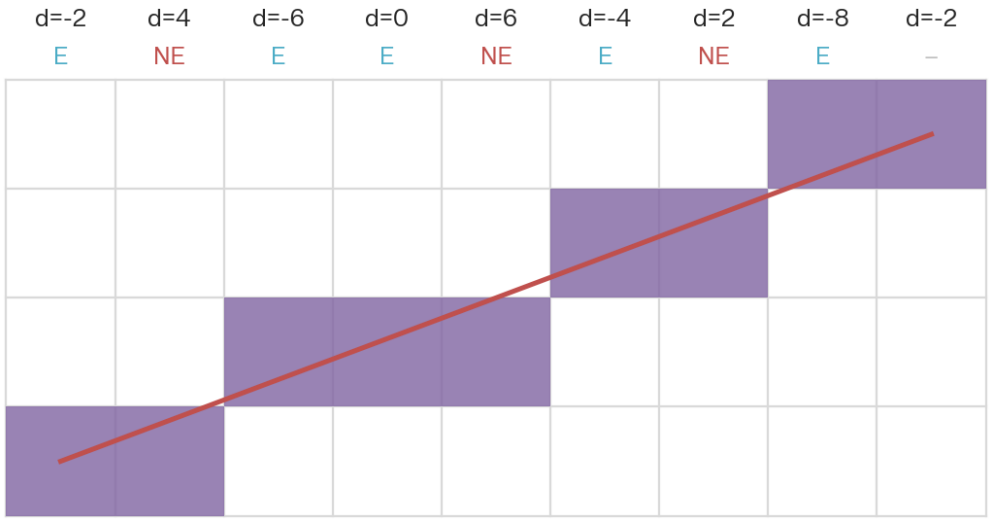
Bresenham: The Algorithm (slope 0 to 1)

Integer-only; the core of every software line routine

```
function lineBresenham(x0, y0, x1, y1, color) {   // assumes x0 < x1 and 0 <= slope <= 1
  const dx = x1 - x0, dy = y1 - y0;
  let d = 2 * dy - dx;                 // initial decision value
  let y = y0;
  for (let x = x0; x <= x1; x++) {
    putPixel(img, x, y, ...color);
    if (d > 0) { y++; d += 2 * (dy - dx); }   // NE: step up, error resets
    else       {      d += 2 * dy;        }   // E : stay on the same row
  }
}
```

Worked Example: (0, 0) → (8, 3)

- dx = 8, dy = 3, so d0 = 2·3 − 8 = −2
- E-step: d += 2·dy = +6; NE-step: d += 2·(dy − dx) = −10
- Pixels: (0,0) (1,0) (2,1) (3,1) (4,1) (5,2) (6,2) (7,3) (8,3)
- **Check: the ideal line y = 0.375x passes within half a pixel of every one**

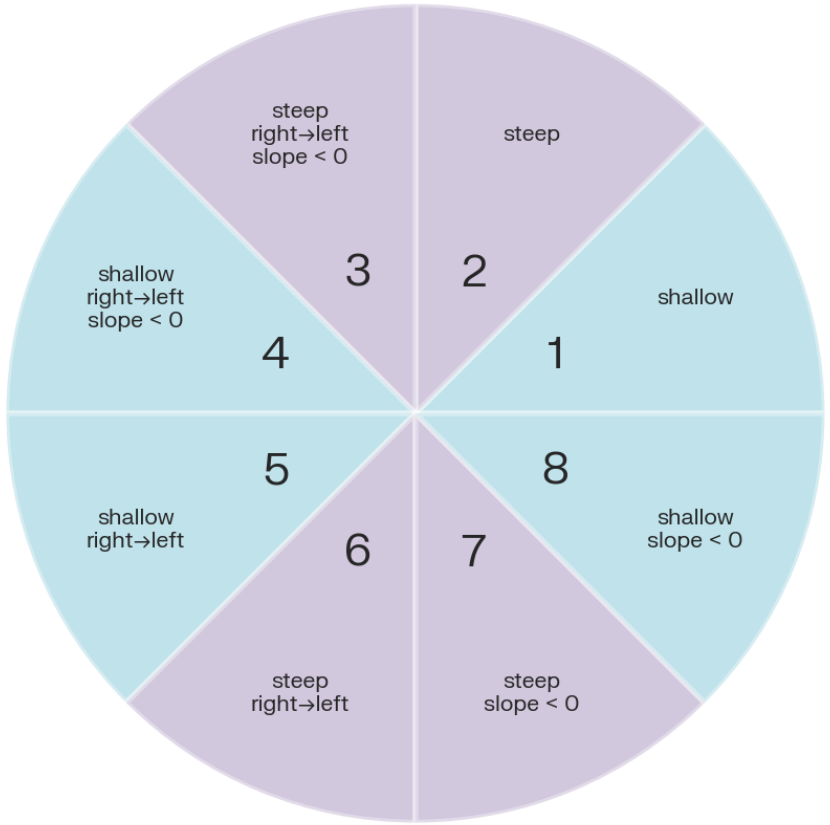


In-Class Check (3 minutes)

- Trace Bresenham for the line (0, 0) → (5, 2).
- 1. What are dx, dy and the initial d?
- 2. Which pixels are plotted?
- 3. Compare with the ideal line y = 0.4·x — how close is each pixel?
- **Think about it yourself, or work with your neighbor**

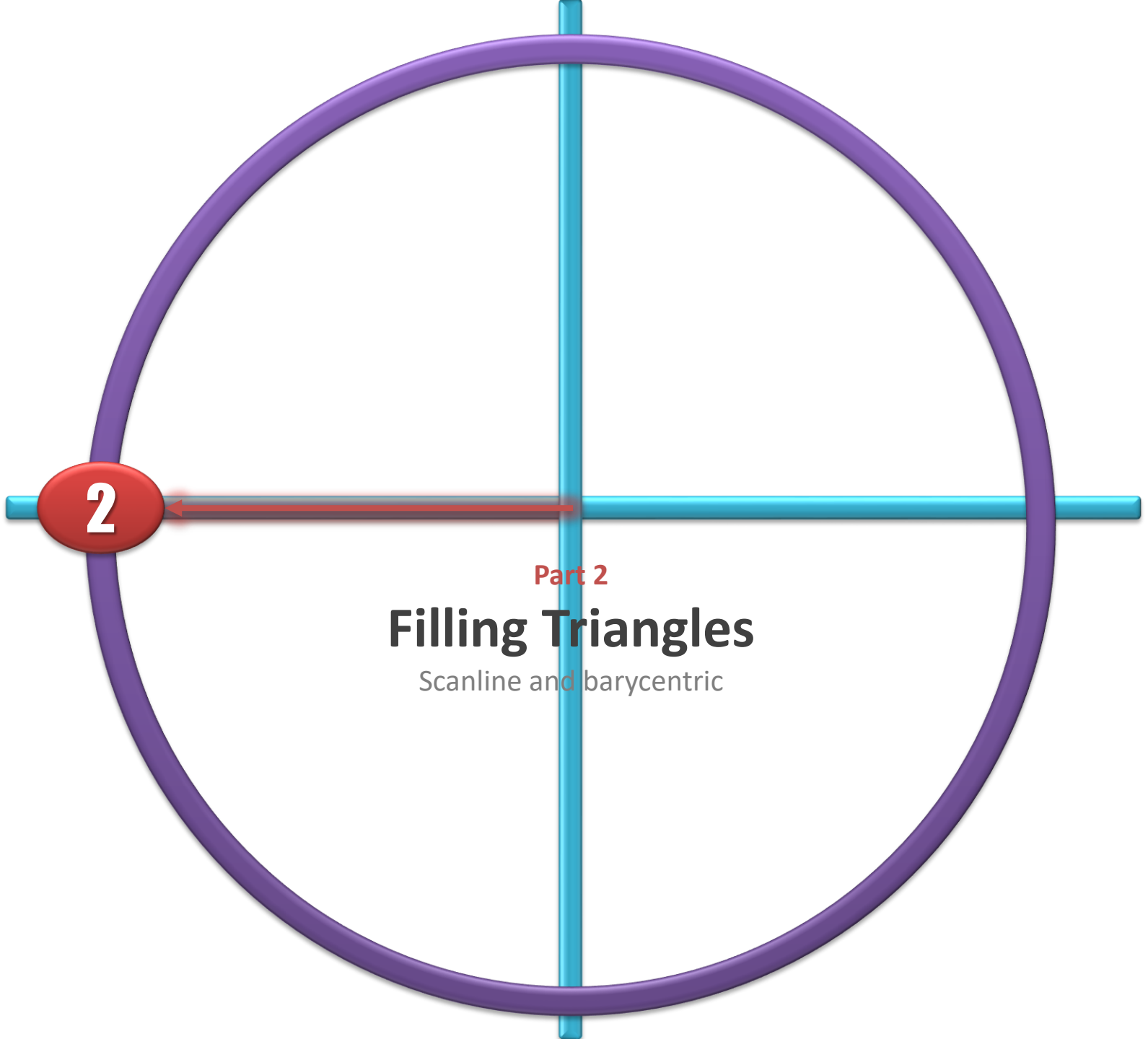
All Eight Octants

- The algorithm so far handles only 0 ≤ slope ≤ 1 and x0 < x1
- Steep lines (|slope| > 1): swap the roles of x and y and loop over y
- Right-to-left lines: swap the two endpoints
- Negative slopes: step y DOWN instead of up
- **Real code reduces all eight cases to the first by symmetry — about six extra lines**



Lines in TinyPipe: Practical Notes

- Clip BEFORE rasterizing (Cohen-Sutherland from W4), or keep the putPixel bounds guard from Lab 1
- Lines from the vertex stage have FLOAT endpoints in screen space: round them once at the start
- Bresenham draws 1-pixel lines; thick or anti-aliased lines need more (coverage, Wu's algorithm)
- GPUs rasterize triangles in the filled pipeline — lines matter for wireframes (your M1 deliverable)



Part 2

Filling Triangles

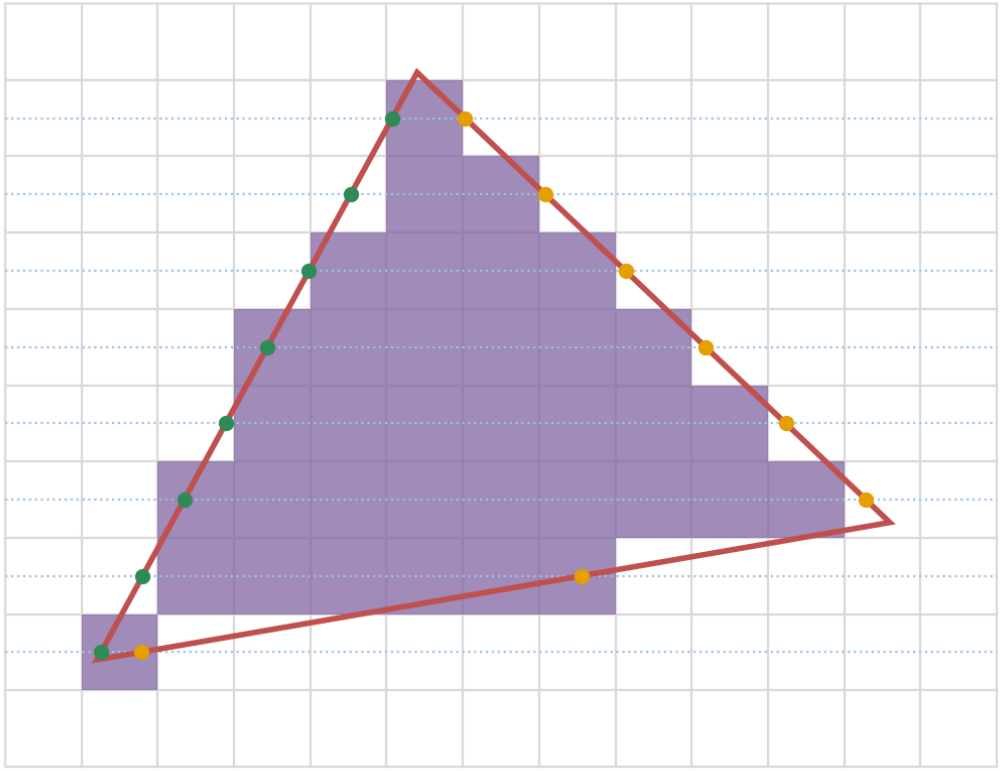
Scanline and barycentric

Why Triangles?

- Every polygon can be split into triangles; GPUs rasterize only triangles
- Triangles are always PLANAR (three points define a plane; a quad can twist)
- Triangles are always CONVEX and SIMPLE (no self-intersection) — the simplest possible fill rules
- One uniform primitive → one hardware pipeline and one set of interpolation rules
- **Two algorithm families: scanline fill (walk rows) and edge functions / barycentric (test pixels)**

Approach A: Scanline Fill

- For each horizontal scanline y from the triangle's top to its bottom:
- 1. Find where the scanline crosses the triangle's edges (x intersections)
- 2. Sort the intersections left to right
- 3. Fill the span between each pair (parity rule: odd crossing = enter, even = leave)
- **For a triangle there are exactly two crossings per scanline: fill from xL to xR**



Edge Table and Active Edge List

- Edge table: for each edge store y_max, x at y_min, and 1/m (= dx/dy)
- Active edge list: the edges crossing the current scanline
- Next scanline: x += 1/m — one ADD per edge per row (the same incremental idea as DDA)
- Remove an edge when y reaches its y_max; add edges when y reaches their y_min
- **The z-buffer's xL / xR / zL / zR interpolation in Part 3 is exactly this edge walking**

Scanline Fill: The Tricky Cases

- Horizontal edges: skip them (no crossing)
- A vertex shared by two edges would be counted twice → use the HALF-OPEN rule: include an edge's y_min scanline, EXCLUDE its y_max scanline
- Fractional x crossings: round xL up and xR down (pixel centres inside the triangle)
- **Bugs here appear as 1-pixel gaps or doubled pixels along shared edges**

Scanline Fill: Triangle Skeleton

Sketch — M2 asks you to write this or the barycentric version

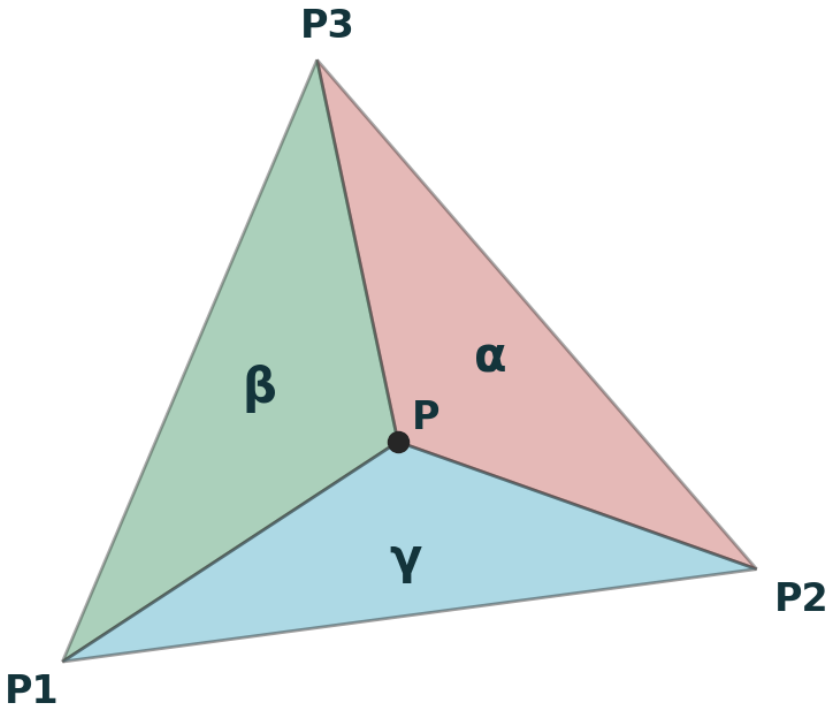
```
function fillTriangleScanline(v0, v1, v2, color) {
  [v0, v1, v2] = [v0, v1, v2].sort((a, b) => a.y - b.y);        // v0.y <= v1.y <= v2.y
  for (let y = Math.ceil(v0.y); y < Math.ceil(v2.y); y++) {      // half-open: exclude the bottom
    const xA = lerpX(v0, v2, y);                                 // long edge spans the full height
    const xB = (y < v1.y) ? lerpX(v0, v1, y) : lerpX(v1, v2, y); // upper or lower short edge
    const [xL, xR] = xA < xB ? [xA, xB] : [xB, xA];
    for (let x = Math.ceil(xL); x < Math.ceil(xR); x++) putPixel(img, x, y, ...color);
  }
}
// lerpX(p, q, y) = p.x + (y - p.y) * (q.x - p.x) / (q.y - p.y)   (guard q.y == p.y)
```

Approach B: Test Every Pixel in the Bounding Box

- Compute the triangle's axis-aligned bounding box
- For each pixel in the box ask: is this pixel inside the triangle?
- Answer with barycentric coordinates — three numbers (α, β, γ) per pixel
- Inside $\Leftrightarrow$ $\alpha \geq 0$, $\beta \geq 0$ and $\gamma \geq 0$
- **Simple, parallel, and how GPUs think: every pixel's test is independent (the Week 2 SIMD idea)**

Recap: Barycentric Coordinates

- Problem: how do we express an interior point P of a triangle in terms of its vertices P1, P2, P3?
- A coordinate system needs an origin and an inside test; barycentric needs neither
- $P = \alpha \cdot P1 + \beta \cdot P2 + \gamma \cdot P3$ with $\alpha + \beta + \gamma = 1$
- Inside $\Leftrightarrow$ all weights ≥ 0
- Generalizes to the convex hull: $P = \Sigma\, \alpha_i \cdot P_i$ with $\Sigma\, \alpha_i = 1$ and $\alpha_i \geq 0$



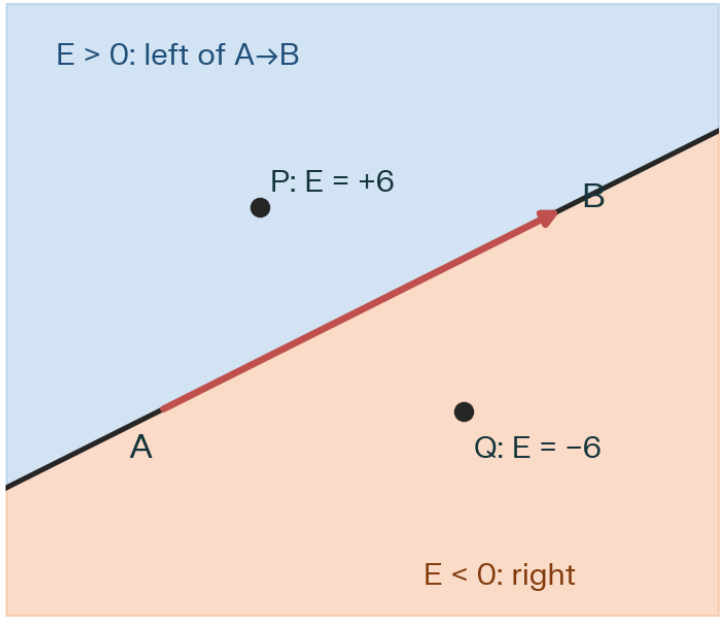
$P = \alpha \cdot P1 + \beta \cdot P2 + \gamma \cdot P3, \quad \alpha + \beta + \gamma = 1$

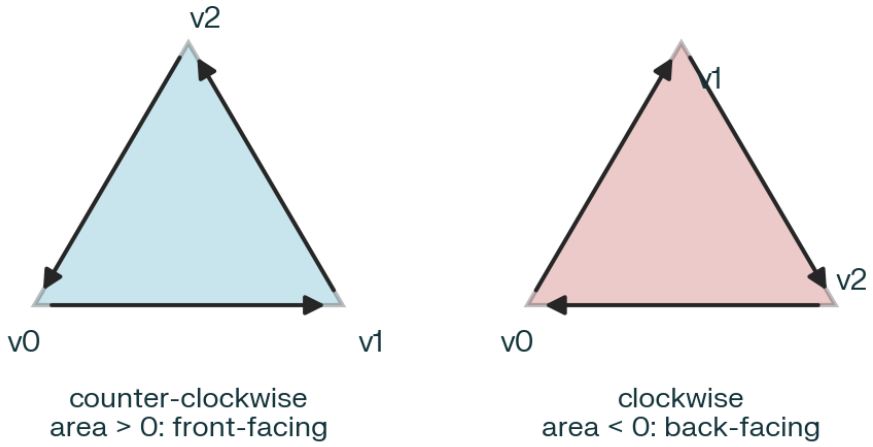
each weight = sub-triangle area / total area

Computing Barycentric Coordinates

- α = area(P, P2, P3) / area(P1, P2, P3), β = area(P1, P, P3) / area(...), $\gamma = 1 - \alpha - \beta$
- Signed area via the EDGE FUNCTION: E(A, B, P) = (B.x − A.x)(P.y − A.y) − (B.y − A.y)(P.x − A.x)
- E > 0: P is left of A→B; E < 0: right; E = 0: on the line (the same sign idea as Bresenham's F)
- α = E(P2, P3, P) / E(P1, P2, P3), β = E(P3, P1, P) / E(P1, P2, P3), γ = E(P1, P2, P) / E(P1, P2, P3)
- **The denominator is twice the signed area: its sign gives the triangle winding**

Edge Function and Winding





Axes drawn y-up; with y-down pixel coordinates every sign flips — pick one convention and keep it.

E(A,B,P) = (B.x − A.x)(P.y − A.y) − (B.y − A.y)(P.x − A.x): the sign tells which side of the line P is on, the same test as Bresenham's F.

The sign of the whole triangle's area is its winding, which back-face culling uses later in Part 3.

Barycentric Triangle Fill

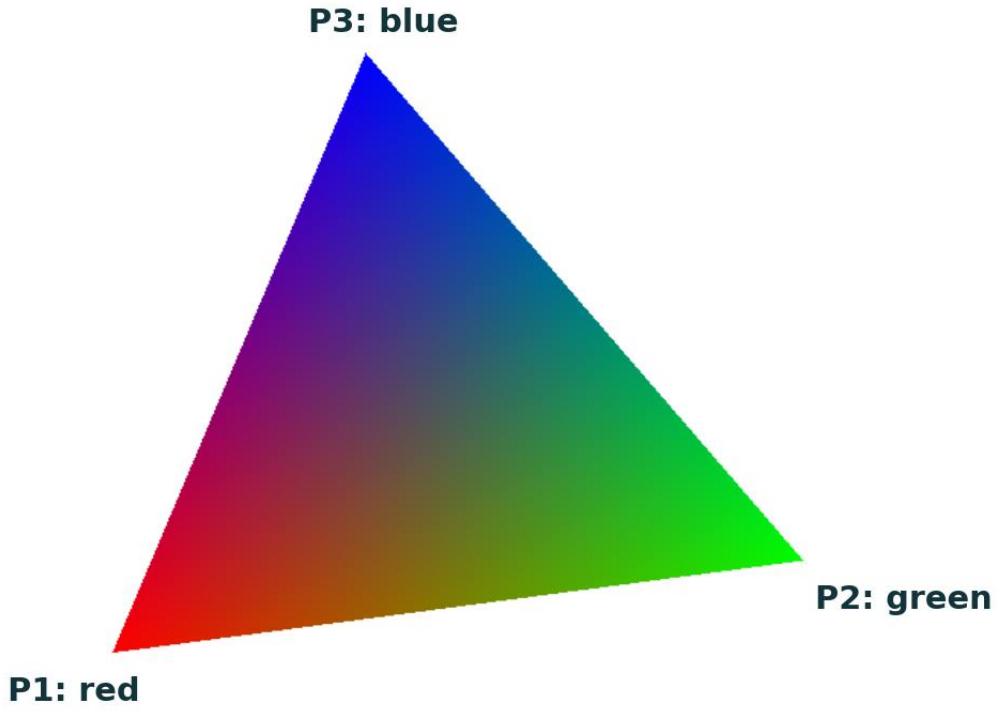
Bounding box + edge functions — the core of M2

```
function edge(a, b, p) { return (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x); }

function fillTriangleBary(v0, v1, v2, color) {
  const area = edge(v0, v1, v2);
  if (area === 0) return;                                    // degenerate triangle
  const minX = Math.max(0, Math.floor(Math.min(v0.x, v1.x, v2.x)));
  const maxX = Math.min(W - 1, Math.ceil(Math.max(v0.x, v1.x, v2.x)));
  const minY = Math.max(0, Math.floor(Math.min(v0.y, v1.y, v2.y)));
  const maxY = Math.min(H - 1, Math.ceil(Math.max(v0.y, v1.y, v2.y)));
  for (let y = minY; y <= maxY; y++)
    for (let x = minX; x <= maxX; x++) {
      const p = { x: x + 0.5, y: y + 0.5 };                  // pixel CENTRE
      const a = edge(v1, v2, p) / area, b = edge(v2, v0, p) / area, c = edge(v0, v1, p) / area;
      if (a >= 0 && b >= 0 && c >= 0) putPixel(img, x, y, ...color);
    }
}
```

What Barycentrics Give You for Free

- Any per-vertex attribute can be interpolated: attr = α·A1 + β·A2 + γ·A3
- Colour → Gouraud shading (W11); depth z → the z-buffer (Part 3); (u, v) → textures (W9); normal → Phong shading (W11)
- One mechanism, four weeks of material
- **WARNING: this interpolation is linear in SCREEN space — right for colour and depth, wrong for textures under perspective. On week 9 we will fix it with the 1/w trick**

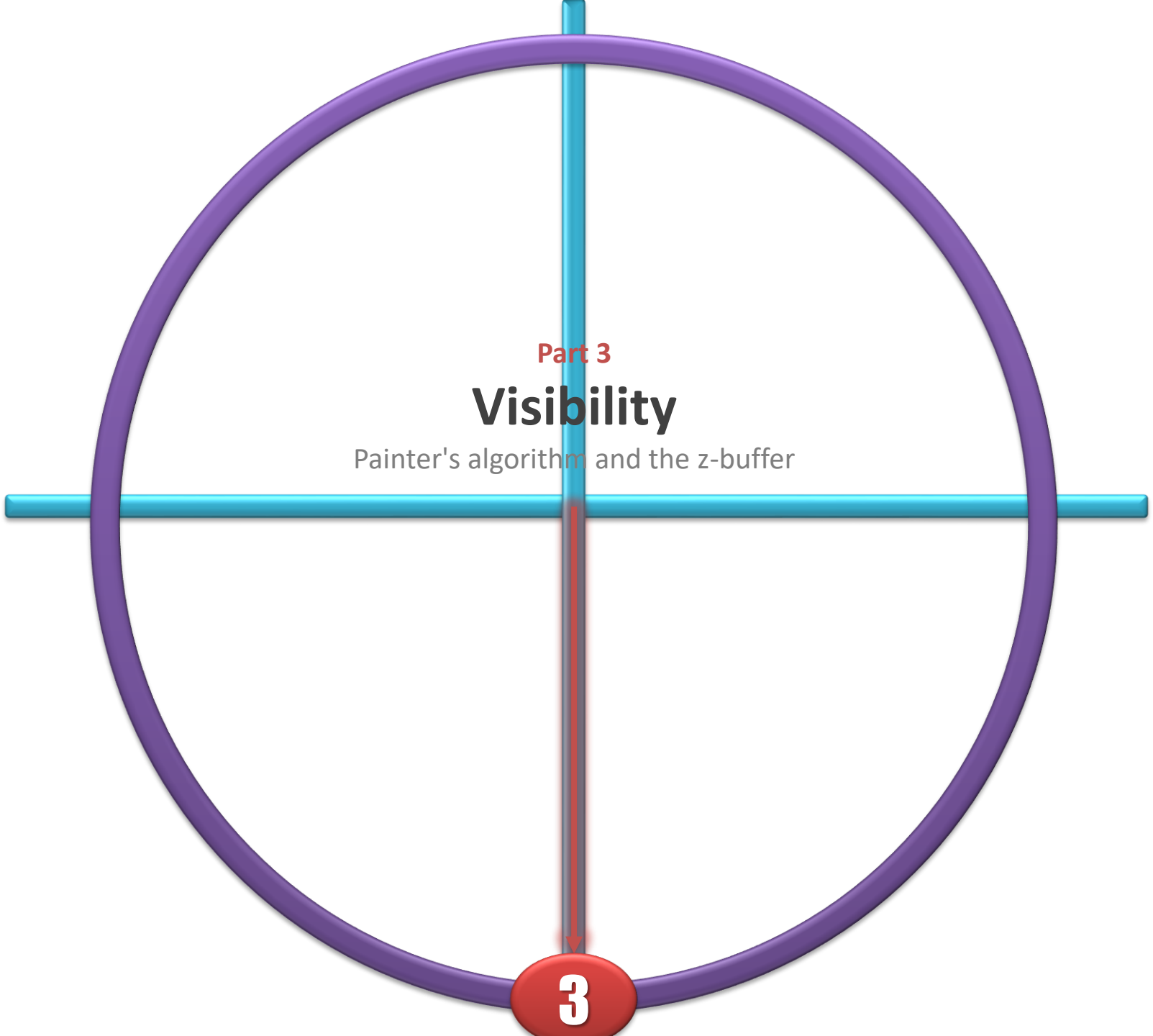


Scanline vs Barycentric

- Scanline — pros: incremental (adds only), touches only covered pixels. Cons: special cases (horizontal edges, shared vertices), harder to parallelize
- Barycentric / edge functions — pros: simple, no special cases, every pixel independent, attributes interpolate directly. Cons: tests pixels in the box that may lie outside (wasteful for thin diagonal triangles)
- Real GPUs: hierarchical edge-function tests over tiles — barycentric thinking at hardware scale

Shared Edges: Draw Each Pixel Once

- Two adjacent triangles share an edge: a pixel exactly ON it must be drawn by exactly ONE of them
- Both draw → double-blended seams (visible with transparency); neither draws → cracks
- GPUs break ties consistently with the TOP-LEFT FILL RULE
- For M2 with opaque colours the simple ≥ 0 test is fine — but know the rule exists


Part 3
Visibility
Painter's algorithm and the z-buffer
3

The Visibility Problem

- Too much to draw = too many cycles: only what the eye can see should reach the screen
- Given two points P1 = (x1, y1, z1) and P2 = (x2, y2, z2):
- 1. Are they on the same projector (line of sight)? 2. If so, which is closer to the eye?
- Different projectors → both visible; same projector → compare depth, the closer one wins

Same Projector: Parallel vs Perspective

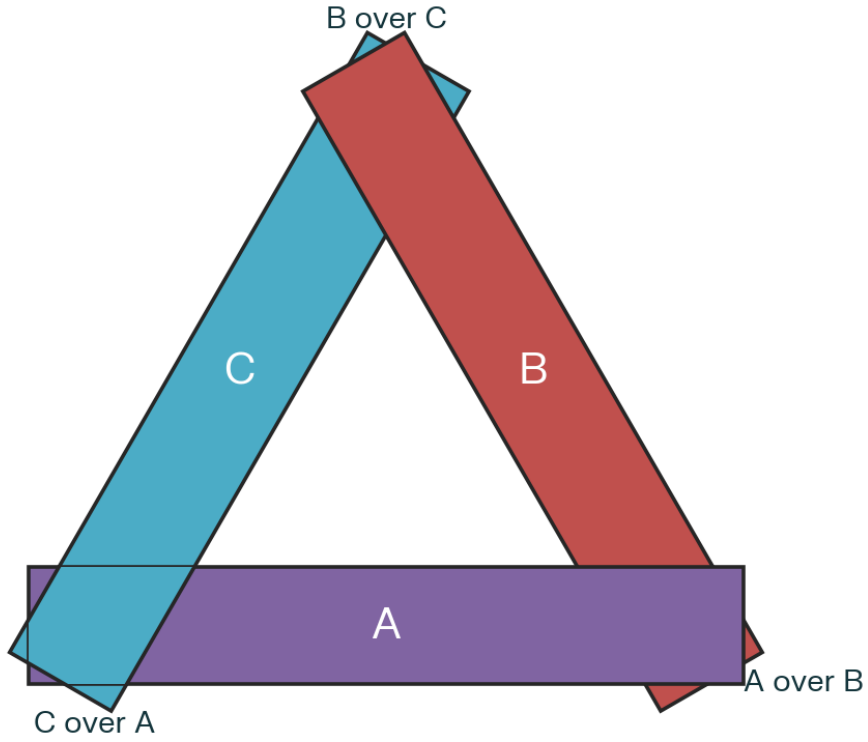
- Parallel projection: same projector $\Leftrightarrow$ x1 = x2 and y1 = y2
- Perspective projection: same projector $\Leftrightarrow$ x1/z1 = x2/z2 and y1/z1 = y2/z2
- **The division by z is the perspective divide from W4 — the same condition in disguise**

Hidden-Surface Algorithms: Two Families

- OBJECT SPACE: compare objects/polygons against each other in 3D, eliminate hidden parts, draw what remains
- IMAGE SPACE: decide visibility independently at each pixel — which surface is closest here?
- Object space: accurate at any resolution, but $O(n^2)$ comparisons and hard cases (cyclic overlap)
- Image space: only as accurate as the pixel grid, but simple and easily parallel
- **GPUs chose image space: the z-buffer**

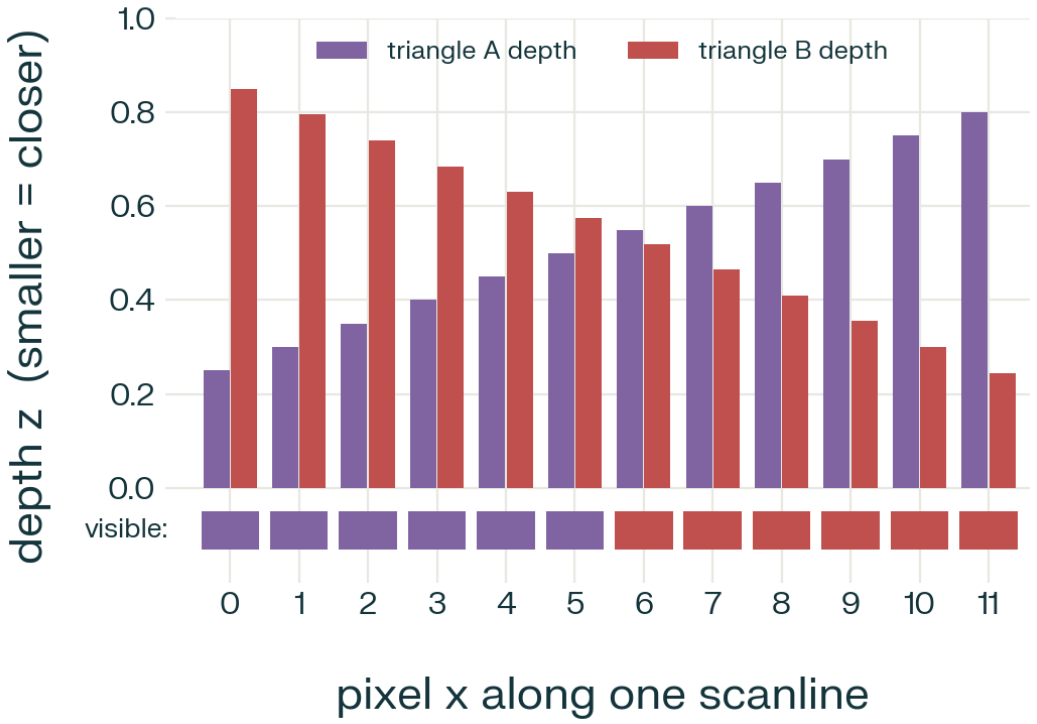
Painter's Algorithm and Its Failure

- Idea: sort polygons far-to-near and paint in that order — nearer ones overwrite farther ones
- Costs a sort every frame, and polygons spanning a range of depths have no single position in the order
- Fails on CYCLIC OVERLAP: A in front of B, B in front of C, C in front of A — no valid order exists
- Fix: split polygons (expensive), or give up ordering and resolve depth per pixel



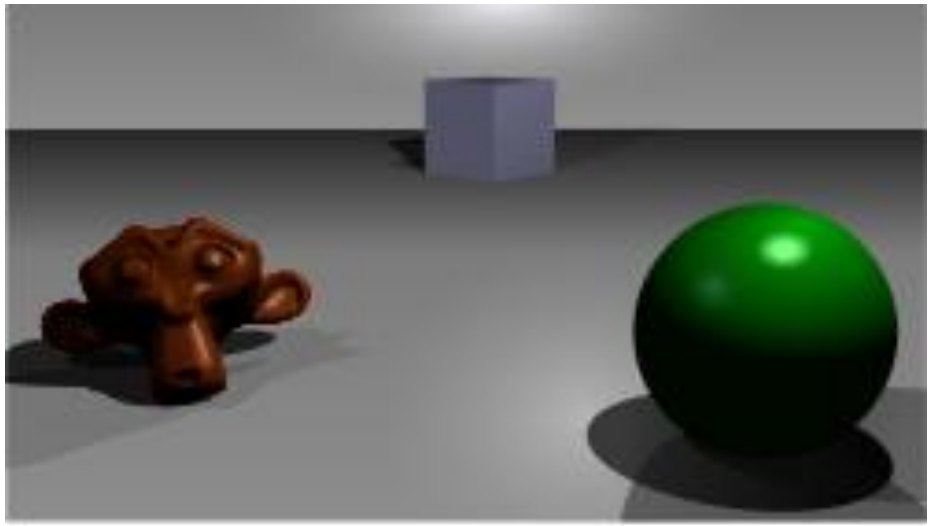
The Z-Buffer Idea

- Keep a second buffer, the same size as the framebuffer, storing the DEPTH of the nearest surface drawn so far at each pixel
- To draw a fragment: compare its depth with the stored depth
- Closer → write colour AND depth; farther → discard
- No sorting, no object comparisons, any draw order, cyclic overlap handled automatically
- **Catmull, 1974 — still the default in every GPU (the depth buffer from W2)**

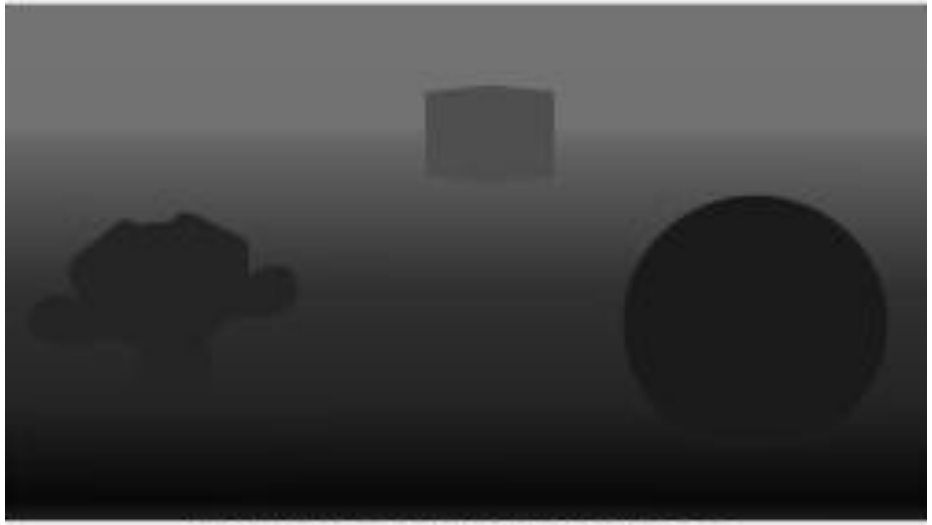


The Z-Buffer

- The image shows a 3D rendered scene with a brown crumpled object, a green sphere, and a gray cube on a flat surface, labeled “A simple three-dimensional scene” above, and below it a darker version labeled “Z buffer representation” that visualizes depth information with shadows and shapes corresponding to the objects.



A simple three-dimensional scene



Z-buffer representation

https://en.wikipedia.org/wiki/Z-buffering

Z-Buffer: The Short Form

- for each polygon:
- for each pixel (x, y) in the polygon:
- newZ = polygonZ(x, y)
- if newZ < zBuffer[x, y]:
- zBuffer[x, y] = newZ; frameBuffer[x, y] = polygonColor(x, y)
- **Convention: smaller z = closer; initialize zBuffer to maxDepth**

How Do We Get z(x, y)?

- A polygon is planar: $A \cdot x + B \cdot y + C \cdot z + D = 0 \rightarrow z(x, y) = -(A \cdot x + B \cdot y + D) / C$
- Depth coherence: one pixel to the right changes z by a CONSTANT: $z(x + 1, y) = z(x, y) - A/C$
- So depth along a scanline costs one ADD per pixel — the same incremental trick as DDA and Bresenham
- **In screen space after the perspective divide, NDC z is linear, so barycentric interpolation of z is exactly right**

Interpolating z Along the Scanline

- At scanline y_s interpolate depth on the two active edges:
- $z_L = z_1 + (z_2 - z_1)\cdot(y_s - y_1)/(y_2 - y_1)$ $\quad z_R = z_1 + (z_3 - z_1)\cdot(y_s - y_1)/(y_3 - y_1)$
- Then across the span: $z_I = z_L + (z_R - z_L)\cdot(x_I - x_L)/(x_R - x_L)$
- Colour c is interpolated the same way — one loop interpolates x, z and c together

The Full Z-Buffer Algorithm

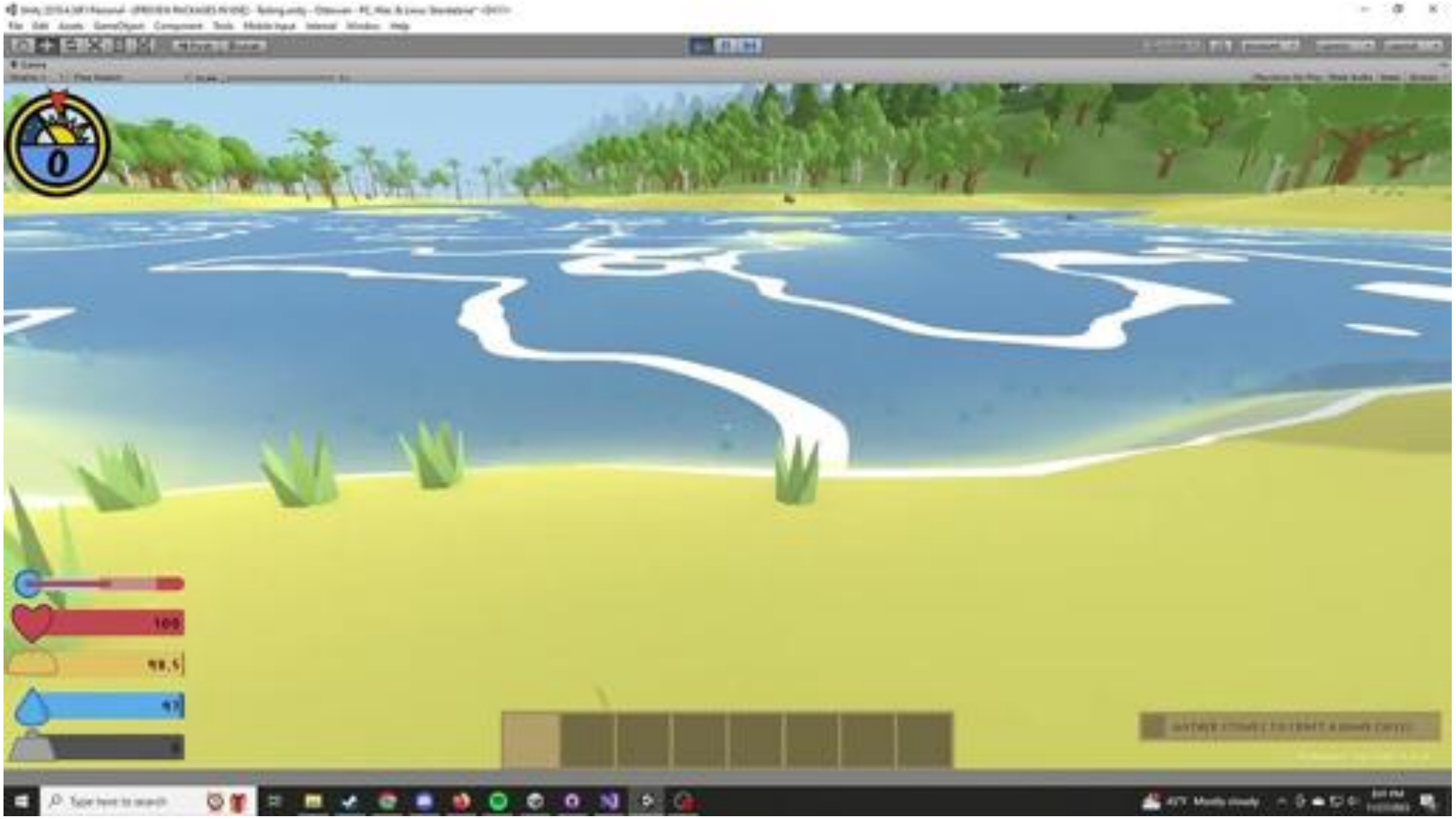
- Initialize: for every pixel, zBuffer[x, y] = maxDepth (+∞, or 1.0 in NDC)
- For each polygon and each scanline y from y_min to y_max: build the edge list
- For each span: interpolate xL, xR, zL, zR, cL, cR
- Inner loop over x: z = interpolate(zL, zR); c = interpolate(cL, cR); if z < zBuffer[x, y]: write z and c
- **Clear the depth buffer every frame — forgetting is the classic "ghost trails" bug (DEPTH_BUFFER_BIT in Lab 4)**

Z-Buffer in Code (CPU)

Extends the barycentric fill — this is M2's finish line

```
const zBuf = new Float32Array(W * H);
function clearDepth() { zBuf.fill(Infinity); }

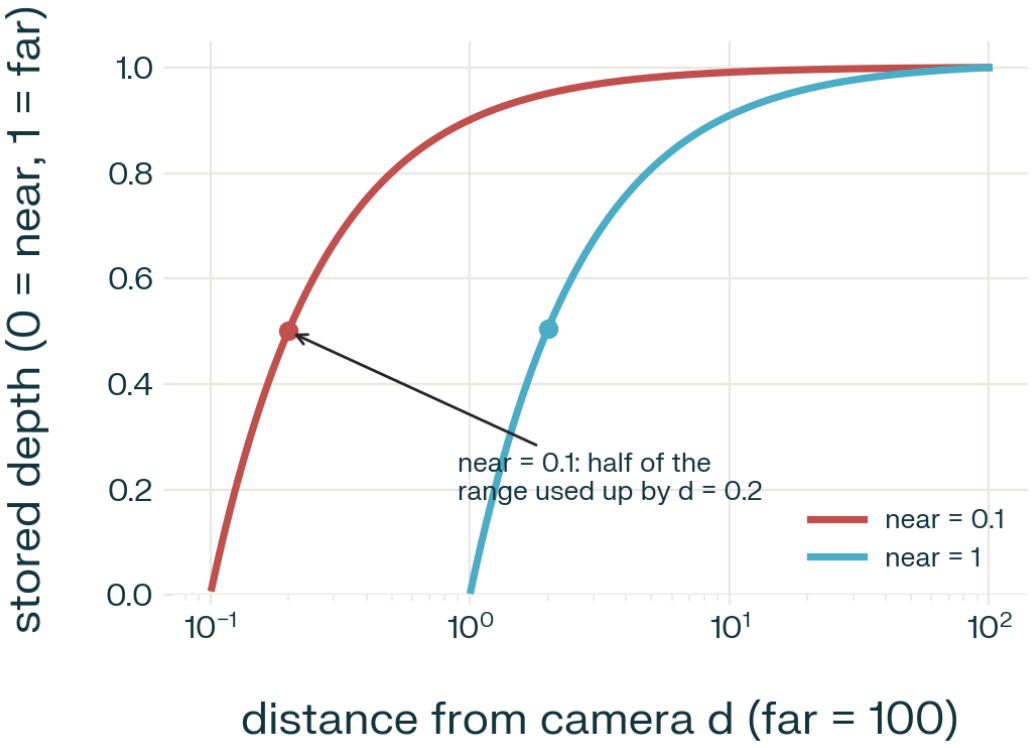
function fillTriangleZ(v0, v1, v2, color) {        // v.x, v.y in pixels, v.z = NDC depth
  const area = edge(v0, v1, v2);  if (area === 0) return;
  /* ... bounding box as before ... */
  for (let y = minY; y <= maxY; y++)
    for (let x = minX; x <= maxX; x++) {
      const p = { x: x + 0.5, y: y + 0.5 };
      const a = edge(v1, v2, p) / area, b = edge(v2, v0, p) / area, c = edge(v0, v1, p) / area;
      if (a < 0 || b < 0 || c < 0) continue;       // outside the triangle
      const z = a * v0.z + b * v1.z + c * v2.z;    // barycentric depth
      const i = y * W + x;
      if (z < zBuf[i]) { zBuf[i] = z; putPixel(img, x, y, ...color); }
    }
}
```



A screenshot of a 3D video game showing a body of water with visible z-fighting artifacts appearing as white jagged lines and flickering patterns on the water surface, set in a landscape with a grass, trees, and a user interface displaying health, hunger, and other stats. https://www.youtube.com/watch?v=o7Gfmxg3tVM

Depth Precision and Z-Fighting

- A depth buffer has finite precision (typically 24 bits) and NDC depth is NONLINEAR in eye distance
- Most precision sits NEAR the near plane; far away, different depths collapse to the same stored value
- Z-fighting: two nearly coplanar surfaces flicker as the tie breaks differently per pixel
- **Fix: push the near plane out (0.1 → 1.0 helps enormously) and keep the far/near ratio modest — the W4 lab's near/far exercise, with consequences**



Z-Buffer: Strengths and Limits

- Strengths: simple, order-independent, any geometry, parallel, hardware-friendly
- Costs: extra memory (W × H × bytes), one compare per fragment, and OVERDRAW — shading hidden fragments is wasted work
- Cannot handle TRANSPARENCY alone: blended surfaces need back-to-front order (the painter's algorithm returns)
- Modern GPUs mitigate overdraw with early-Z tests before running the fragment shader (W6)

Back-Face Culling: Skip the Obvious

- A closed solid's far-side triangles are never visible — don't rasterize them
- Winding order: counter-clockwise on screen = front-facing (convention); clockwise = back-facing
- The sign of the triangle's signed area (our edge function denominator) gives the winding
- WebGL: gl.enable(gl.CULL_FACE) culls back faces — roughly half the triangles of a closed mesh
- **Decide AFTER projection: judging by the 3D normal alone is subtly wrong (see the W12 modelling notes)**

Project hint, End to End

- 1. Vertices → model/view/projection matrices (M1) → perspective divide → viewport → screen-space (x, y) + depth z
- 2. For each triangle: bounding box → per-pixel barycentric test (or scanline fill)
- 3. Per covered pixel: depth test against the z-buffer → write colour and depth
- 4. Each frame: clear colour and depth, redraw

Lab 4 Connection: Meet the GPU's Z-Buffer

- Lab 4: draw an indexed cube with gl.drawElements, then toggle gl.DEPTH_TEST on and off
- Compare the hardware depth test with the CPU z-buffer you just wrote: the same algorithm, orders of magnitude faster
- Disable depth testing and watch painter's-order artifacts appear — "Painter's Algorithm and Its Failure" made visible

Week 5: Takeaways

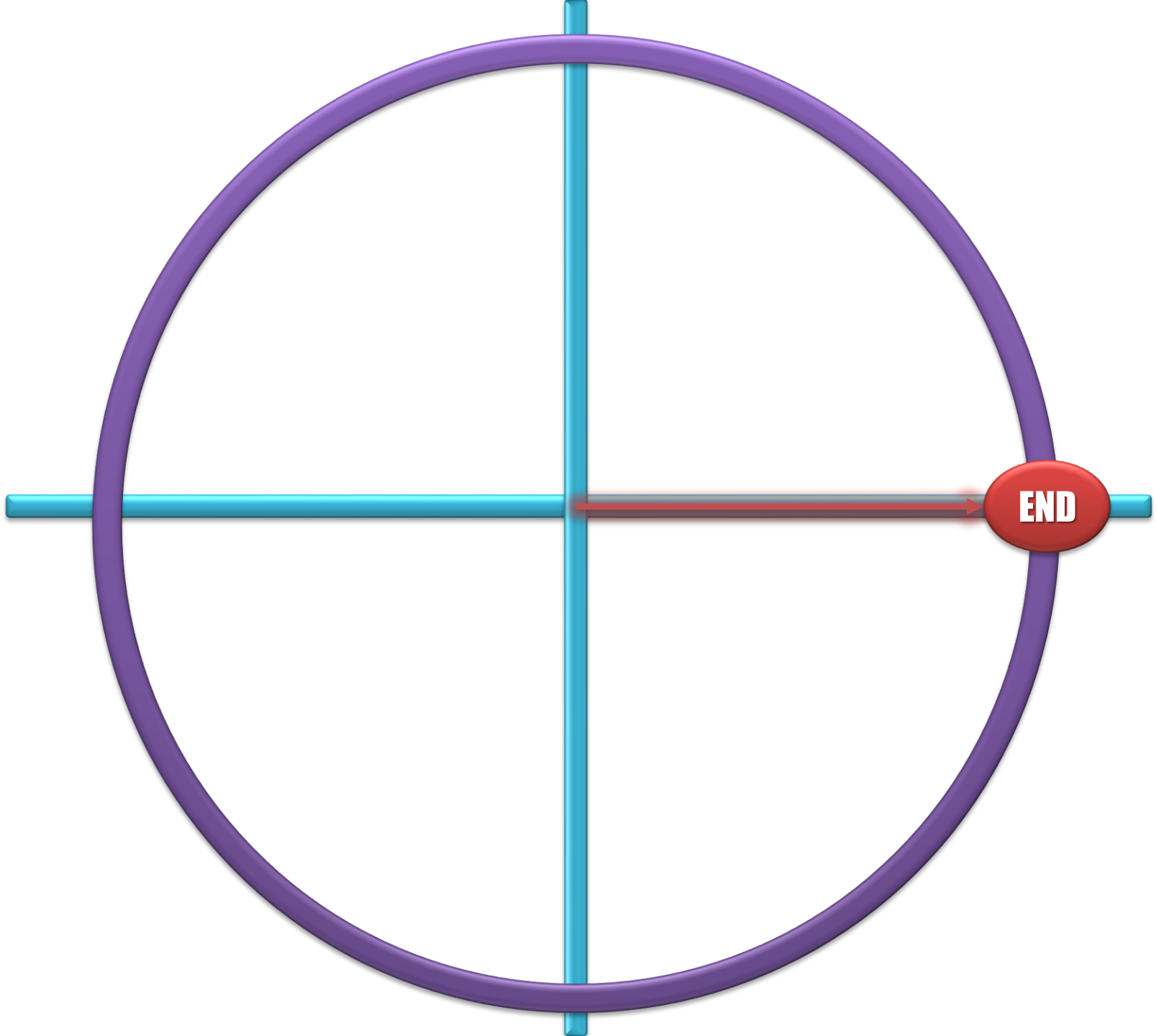
- Rasterization turns continuous geometry into pixels: DDA adds, Bresenham decides with integers
- Triangles fill by scanline (walk rows) or barycentric/edge functions (test pixels); barycentrics also interpolate every attribute
- Visibility is solved per pixel by the z-buffer: compare depth, keep the nearest

Next Week (W6): Shaders

- Today the colour of a pixel was a constant or an interpolated value. Next week a PROGRAM computes it — the fragment shader
- The vertex and fragment stages become programmable: GLSL ES 3.00 in WebGL2
- **Lab 5: shader playground + the CPU-vs-GPU benchmark — you will time your own CPU loops against the GPU**

References & Further Reading

- Marschner & Shirley, Fundamentals of Computer Graphics (5th ed.) — the graphics pipeline, rasterization and barycentric interpolation
- Scratchapixel — "Rasterization: a practical implementation" (free)
- ssloy/tinyrenderer — lessons on Bresenham, triangle rasterization and the z-buffer (free, ~500 lines)
- Akenine-Möller et al., Real-Time Rendering (4th ed.) — rasterization and depth-buffer chapters; realtimerendering.com
- CMU 15-462 (Keenan Crane) — "Drawing a Triangle" and "Depth and Transparency" lectures (public slides)


END